

Arrays in Bash (Terminal Style)

Definition:

An array in Bash is a variable that can hold multiple values at once, accessible using an index number.

1. Indexed Arrays

These use numeric indexes (0, 1, 2...).

Example:

```
fruits=("apple" "banana" "cherry")
echo ${fruits[0]}    # apple
echo ${fruits[@]}    # all elements
echo ${#fruits[@]}   # array length
```

Loop Example:

```
for item in "${fruits[@]}; do
    echo $item
done
```

Output:

```
apple
banana
cherry
```

2. Associative Arrays

These use string keys (like dictionaries). Must be declared with `declare -A`.

Example:

```
declare -A capital
```

```
capital[France]="Paris"
capital[Japan]="Tokyo"
echo ${capital[France]}
```

Loop Example:

```
for country in "${!capital[@]}"; do
    echo "$country -> ${capital[$country]}"
done
```

Output:

```
France -> Paris
Japan -> Tokyo
```

3. User Input Arrays

You can take user input and store it in arrays.

Example (Indexed):

```
read -p "Enter three fruits (space separated): " -a fruits
echo "You entered: ${fruits[@]}"
```

Example (Associative):

```
declare -A country
read -p "Enter country: " c
read -p "Enter capital: " cap
country[$c]=$cap
echo "Capital of $c is ${country[$c]}"
```

Output Example:

```
Enter country: Japan
Enter capital: Tokyo
Capital of Japan is Tokyo
```

Summary Table

Type	Declaration	Index Type	Example Access
-----	-----	-----	-----
Indexed Array	Optional	Numeric	\${arr[0]}
Associative	declare -A	String	\${arr[key]}
User Input	read -a arr	Numeric	\${arr[@]}